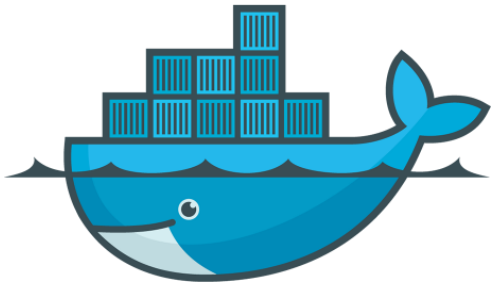
Infrastructure as Code

Containers

Ron C. Chiang
University of St. Thomas



Agenda

- Containers
  - Purpose and benefits
  - Containers vs. Virtual Machines
- Docker
  - Overview
  - Docker client commands
  - Dockerfile
  - Images
  - Networking
  - Volumes

Docker Hands-on

- Feel free to try the Docker command examples throughout the lecture:
  - https://seis665-public.s3.amazonaws.com/docker-single-server.json
  - Launches a stack containing a single Ubuntu Linux instance with the Docker engine software installed.
  - Remember to log into the instance using the "ubuntu" user name.

Fragile IT

Many different application stacks



Static website

Nginx + bootstrap + jekyll



Queue

Redis + redis-sentinel



Analytics

Hadoop + pig + spark



Web API

Python + flask + celery + postgres



Many different hardware environments



Development Virtual machine



QA server



Public cloud



Production servers



Developer laptop

The Matrix from Hell



Static website	?	?	?	?	?



Queue	?	?	?	?	?



Analytics	?	?	?	?	?



Web API	?	?	?	?	?
	Development VM 	QA Server 	Public Cloud 	Production servers 	Developer laptop 

Transporting Cargo Pre-1960

Many different products

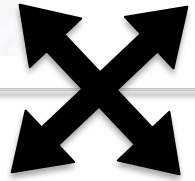






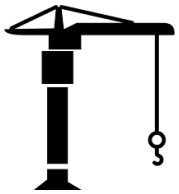






Many different transport methods











Cargo Transport Matrix

	?	?	?	?	?
	?	?	?	?	?
	?	?	?	?	?
	?	?	?	?	?
	?	?	?	?	?
					

Intermodal Transport Solution











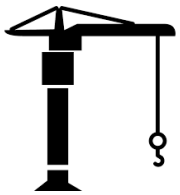
Intermodal shipping container

- Shipping container sizes standardized
- 90% of all cargo shipped in standard containers



- Can ship cars next to beans
- Don’t need to worry about transitioning from ship to truck
- 5000 ships deliver 200M containers per year











Intermodal Shipping Containers









IT Transport Solution: Containers



Static website



Queue



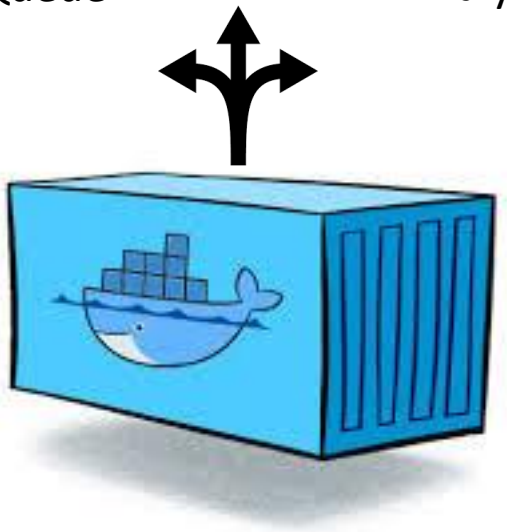
Analytics



Web API

A container is the IT equivalent of a shipping container.

- Encapsulates application stack in a lightweight and portable environment



- Can be manually composed using simple operations
- Runs on almost any hardware platform using a modern OS kernel



Development Virtual machine



QA server



Public cloud

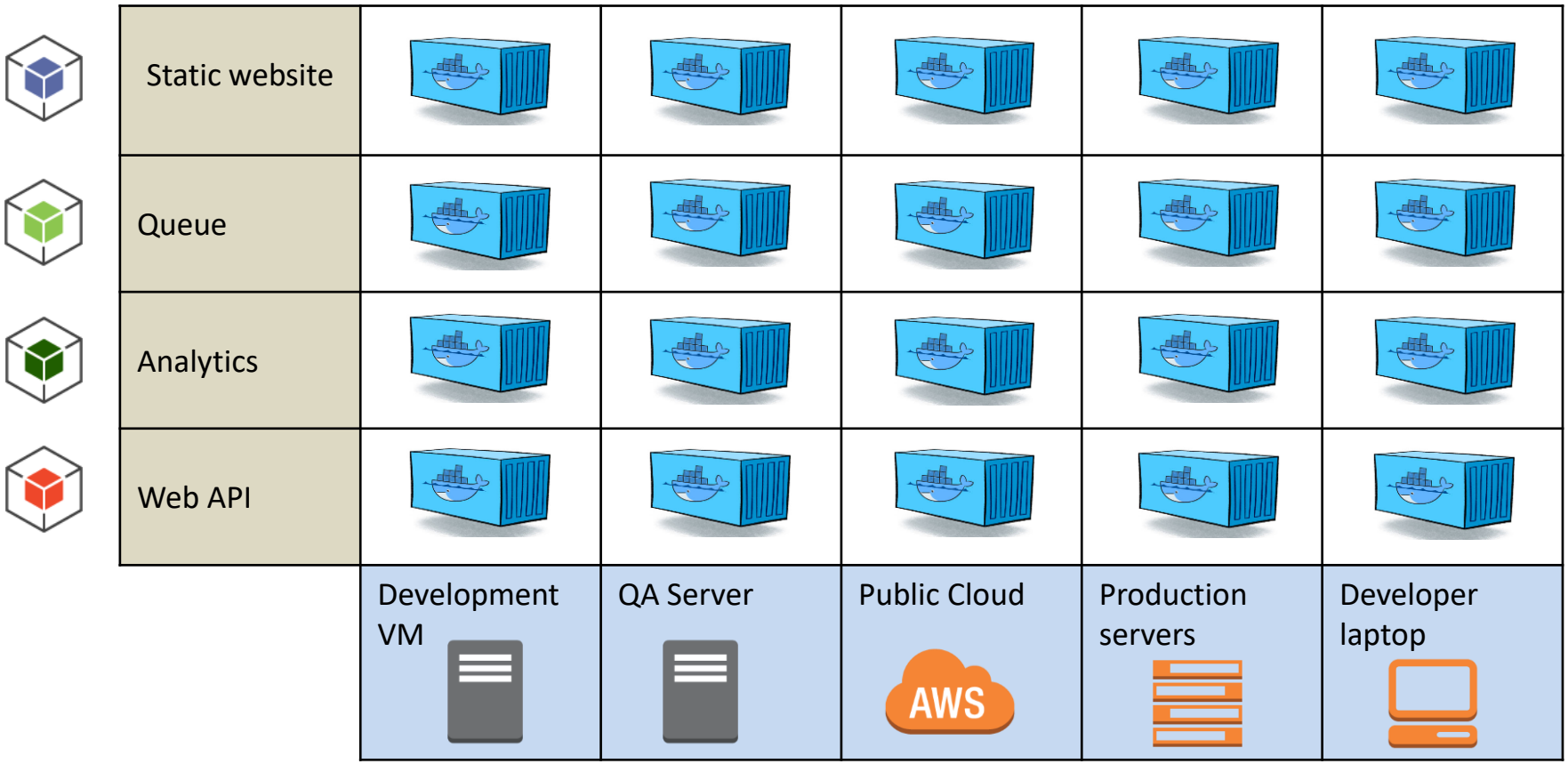


Production servers



Developer laptop

IT Matrix Solution



Developer Benefits

- **Build the application stack once and run it anywhere.**
  - Eliminate concerns about compatibility on different hardware platforms.
    - The application will run in dev, testing, and at the customer site.
  - No worries about missing application dependencies like required packages.
  - Each application runs in its own container to eliminate conflicts like different shared libraries.
  - Easy to automate application testing and deployments using scripting.
  - Much less performance penalty than virtual machines.

IT Operations benefits

- **Configure the application once, run it anywhere.**
  - Application lifecycle (develop/test/deploy) becomes much more repeatable
  - Eliminate inconsistencies between development, testing, and production environments
  - Simplifies continuous integration/deployment practices
  - Addresses performance and financial costs associated with virtual machines

High Level Overview

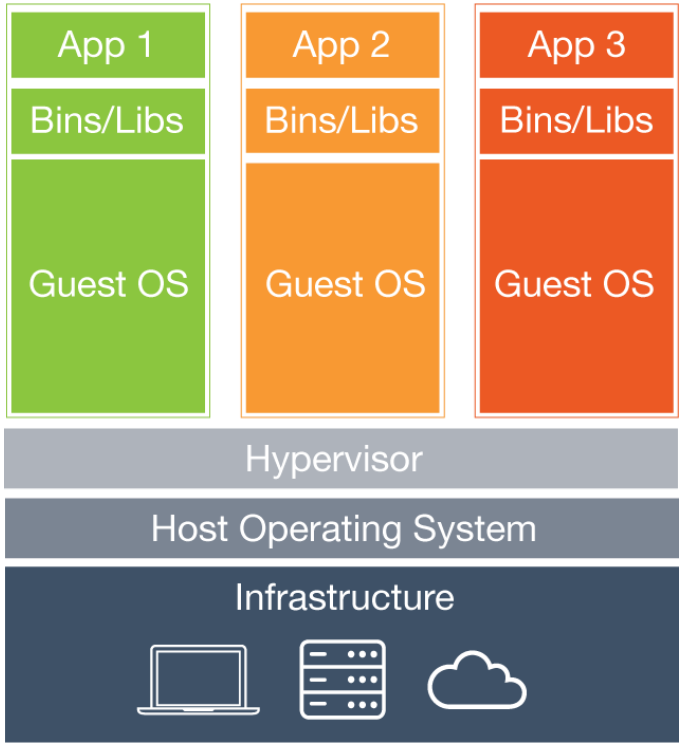
- A container feels like a lightweight VM.
  - You can log into it and get a shell.
  - It has its own process space.
  - You can run commands within the container as root.
  - You can install packages in the container.
  - The container can run services.
  - You can modify network settings in the container.

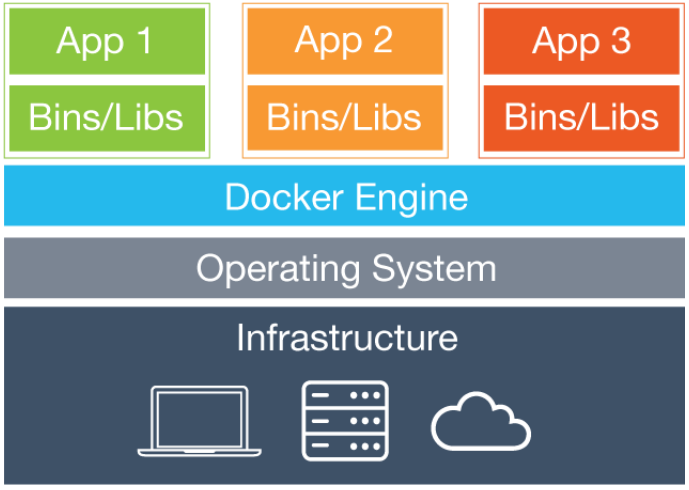
Low Level Overview

- But… it's not quite like a virtual machine.
  - It uses the host kernel.
  - It can't run a different OS than the host.
  - It can't load its own kernel modules.
  - It can't emulate devices like usb, storage, etc.
  - Network using port forwarding
  - It doesn't need to run OS-type processes like syslogd and cron.
  - It's like chroot on steroids.
    - chroot = process jail that prevents the process from accessing external files outside an "apparent root directory"

Virtual Machines vs. Containers

Key difference: Each virtual machine contains an OS whereas all containers share the same OS.

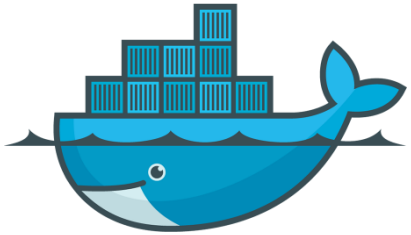




Container advantages over VMs

- What can a container do that a VM cannot?
  - Really simple app deployment with a single command line.
  - “Boot” in a fraction of a second compared to a minute for a VM.
  - Enables very granular services (micro-service architecture).
  - Significantly reduce testing and development footprint.
    - Imagine the resource savings when launching hundreds of containers vs VMs.

Docker



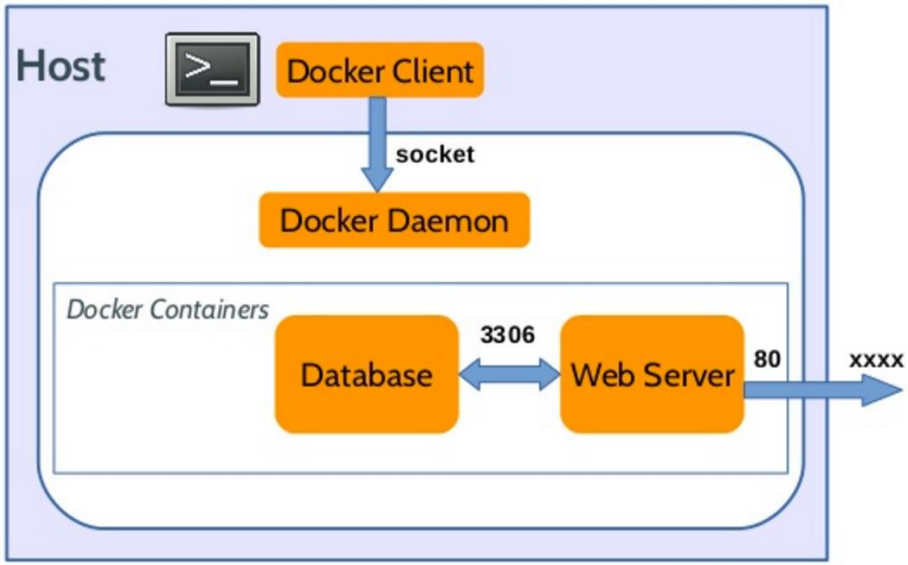
- An open-source Linux-based container implementation started by Solomon Hykes in France.
- Initially released in 2013 and now in widespread use by cloud providers and enterprises.
- It's really an API and toolset that utilizes existing Linux facilities like cgroups and namespaces.
- Docker wasn't the first Linux-based container solution
  - Google developed cgroups 10 years earlier.
  - OpenVZ
  - Virtuozzo

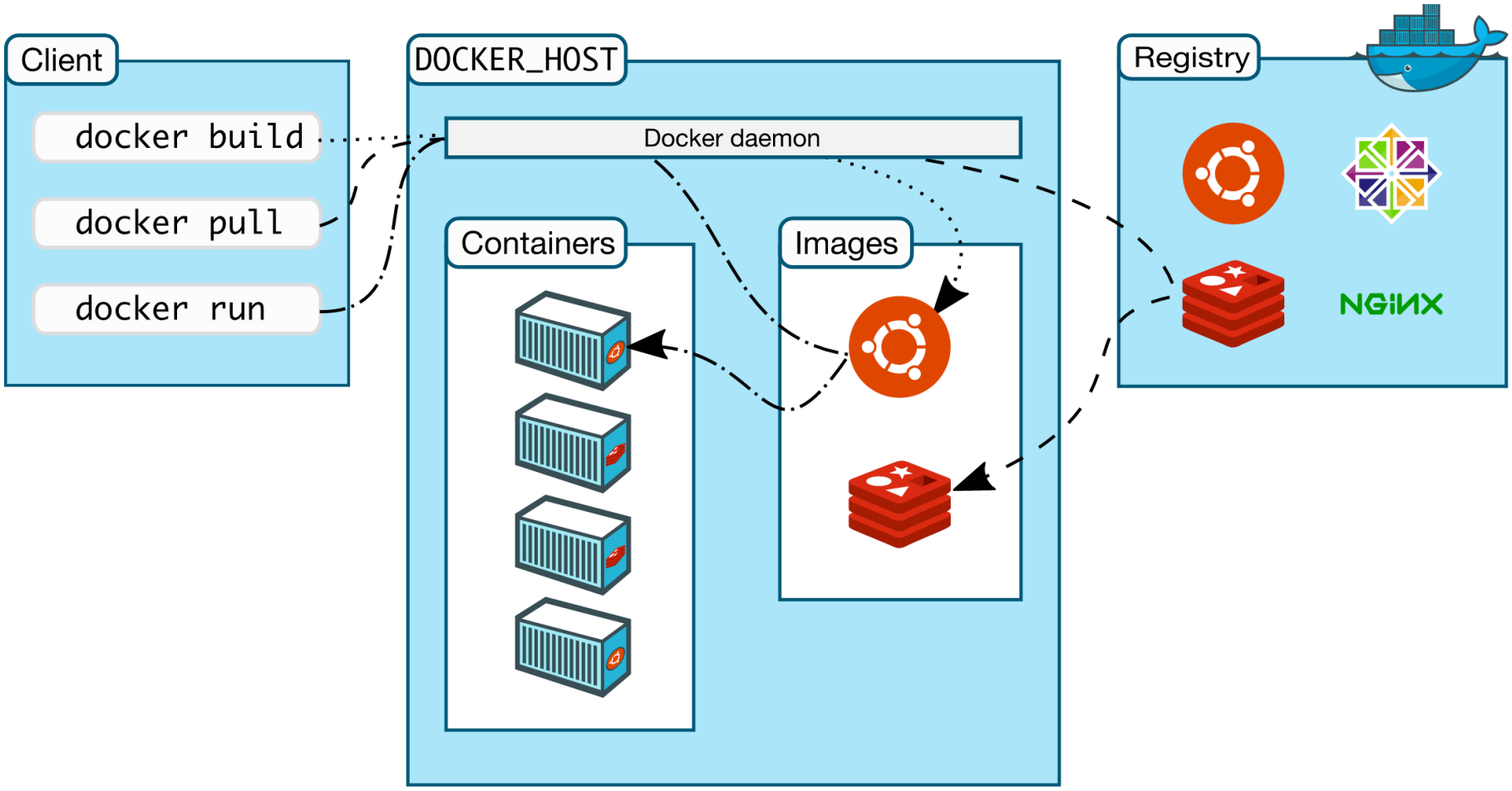
Traditional vs. Docker Application Deployment

- Traditional
  - Ship packages via rpm, gem, jar, etc.
  - Deal with dependency conflicts.
  - “Works on my machine.
- Container
  - Ship containers with all dependencies.
  - Break application stack into layers and only ship layers that change.
  - “Works on any machine.”

How Docker Works

- Implemented as a client-server system.
  - Docker daemon (engine) runs on a host.
    - Daemon builds, runs, and manages containers.
  - Docker CLI is the client app.
    - Takes user input and sends it to the daemon.
    - CLI can run on the host or a different machine.
  - Docker Hub Registry
    - Collection of public images





Start
Stop
Terminate

Docker Terms

- **Host**: a machine running containers.
- **Image**: a read only template containing files and meta-data used to create containers.
- **Container**: an isolated application platform containing everything needed to run an application.
- **Registry**: a repository of images.
- **Volume**: data storage outside the container.
- **Dockerfile**: a script for creating images.

Hello World Example

```
$ docker run busybox echo hello world
hello world
```

- What's happening here?
  - Docker created a small container using the `busybox` image (very small embedded system).
  - Ran a single process and echoed the text "hello world" using the **docker run** command.

Interactive Container

```
$ docker run –it ubuntu bash
root@42b6393c0d26:/#
```

- Start up a minimal ubuntu system.
- -i: connects to the container’s stdin (standard input)
- -t: sets up a pseudo-terminal
- We can install new packages while connected to the container.

Background Container

- Containers can run in the background when started with the –d (detached) flag.

```
$ docker run –d jpetazzo/clock
116f074e161b365441f8444a21318068e987276011
d3c02086f405702c65921b
```

- What's happening?
  - Docker downloads image jpetazzo/clock
  - We don't see the output, but Docker logs it.
  - Docker responds with the ID of the container.

List Containers

- The **`docker ps`** command works just like the Linux ps command and shows running containers.
- Command output:
  - Container ID
  - Image used to create the container
  - How long the container has been running
  - The command running within the container.

```
$ docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED         STATUS        PORTS      NAMES
116f074e161b   jpetazzo/clock  "/bin/sh -c 'while da"  12 minutes ago  Up 12 minutes
peaceful_jones
```

View Container Logs

- Containers running in the background will log output to Docker.
- Use the **`docker logs <id>`** command to view the logs.

```
$ docker logs 116f
Mon Sep  5 03:26:34 UTC 2016
Mon Sep  5 03:26:35 UTC 2016
Mon Sep  5 03:26:36 UTC 2016
```

- Only need to specify the prefix of the container ID.
- Use the **–tail** option to see the most recent # log output.

```
$ docker logs –tail 3 116f
$ docker logs –tail 3 –follow 116f
```

Stopping and restarting the container

- Docker provides two ways to terminate the background container:
  - **`docker stop`**: graceful shutdown via TERM signal.
  - **`docker kill`**: forcibly stops using KILL signal.

$ docker stop 116f
116f

- List all stopped containers using the –a option:

```
$ docker ps –a
```

- Restart the container using the command:

```
$ docker start <container ID>
```

Docker container names

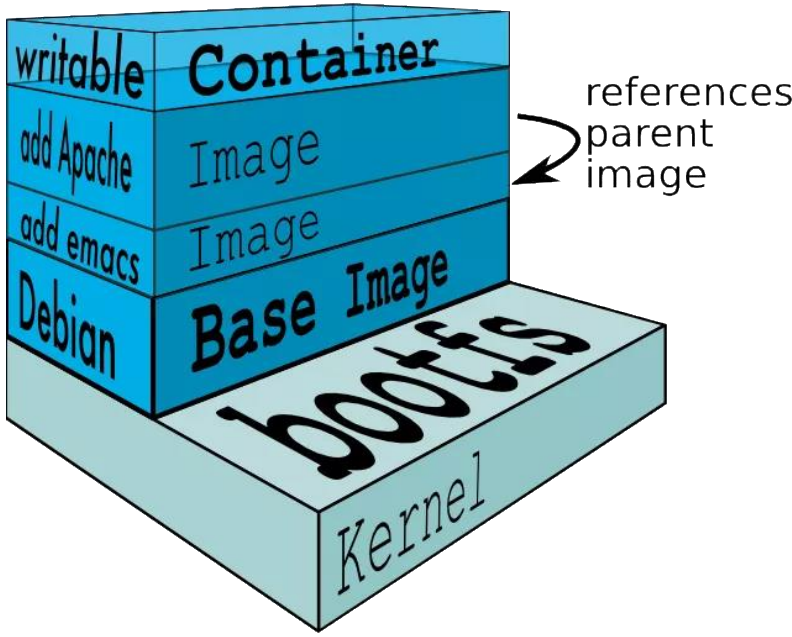
- It's easier to reference a container name than using the container ID.
- Docker automatically assigns names: <mood>_<inventor name>
  - happy_curie
  - clever_hopper
- Specify a name using **`docker run --name <name>`**:

```
$ docker run --name ticktock jpetazzo/clock
```

- Docker will not create a container if the specified container name already exists.
- Rename a container using **`docker rename <oldname> <newname>`**

Docker images

- Image is a collection of files and meta data.
- Images made of up layers stacked on top of each other.
- Each layer can add, change, and remove files.



- Images can share layers to optimize disk, downloading, and memory use.

Images versus Containers

- Image is a read-only filesystem.
- Container is an encapsulated set of processes running a read-write copy of an image.
  - Utilizes copy-on-write to speed up boot time.
- Images are templates you can create containers from.

Modifying Images

- Images are read-only so it's not possible to change an image.

- However… we can do the following:
  1. Create a new container from an image.
  2. Make changes to the container.
  3. Transform the changes into a new layer.
  4. A new image is created by stacking the new layer on top of the old image.

Image namespaces

- Images can have three namespaces:
  - Official images
    - Example: busybox
    - Maintained by Docker Inc. and containing many general utility images.
  - User images
    - Example: jpetazzo/clock
    - The first part denotes the Docker Hub user and the second part specifies the image.
  - Self-hosted images
    - Example: registry.mydomain.com:5000/wordpress
- The docker daemon can push or pull images from a registry (Docker Hub or private repository).

Show current images

- The **`docker images`** command shows what images are located on the host machine.

```
$ docker images
REPOSITORY      TAG             IMAGE ID            CREATED             SIZE
ubuntu          latest          bd3d4369aebc        9 days ago          126.6 MB
hello-world     latest          c54a2cc56cbb        9 weeks ago         1.848 kB
busybox         latest          2b8fd9751c4c        10 weeks ago        1.093 MB
```

- Search for images using the **`docker search`** command:

```
$ docker search hello-world
```

Installing images

- Docker provides two ways to download images onto the host machine:
  - **`docker pull`** : explicitly downloads image
  - **`docker run`** : automatically downloads image if not found on the local host

```
$ docker pull debian:jessie
Pulling repository debian
b164861940b8: Download complete
b164861940b8: Pulling image (jessie) from debian
d1881793a057: Download complete
```

- Docker downloads each layer of the image individually.

Image tags

- Images have tags which define the versions of the image.
  - Different versions of Linux distributions.
  - Different version of application servers.
- If a tag isn't specified then the **:latest** tag is used.
- It's a good idea to specify a tag when the container is used in production.

Build an image

- Start a new container:

```
$ docker run –it ubuntu:bionic bash
```

- Install new packages on the container:

```
root@<id>:#/ apt-get update && apt-get install figlet
```

- Exit the container

- Inspect the container changes by using the **`docker diff`** command:

```
$ docker diff
C /root
A /root/.bash_history
C /tmp
C /usr
C /usr/bin
A /usr/bin/figlet
…
```

Build an image

- Create the new image layer based on the changes using the **`docker commit`** command:

```
$ docker commit <container ID>
```

- The output from the commit command is the new image ID.

```
$docker run –it 48da
root@3db4b8954811:/# figlet hello
```

- We can also tag the image to make it easier to use:

```
$ docker tag <new image ID> figlet
$ docker commit <containerID> figlet
```

Dockerfile

- Building Docker images by hand is tedious and error-prone.
- A Dockerfile provides a recipe to build images using code in a text file.
- The **`docker build -t <tagname> .`** command is used to build an image from a Dockerfile.

```
# Dockerfile
FROM ubuntu
RUN apt-get update -y
RUN apt-get install figlet -y
```

- FROM specifies the base image
- RUN lines are executed by Docker on the container during the build.
- RUN commands must be non-interactive.

Building Dockerfile

```
$ docker build -t figlet .
Sending build context to Docker daemon 2.048 kB
Step 1 : FROM ubuntu ---> bd3d4369aebc
Step 2 : RUN apt-get update -y ---> Running in 6f1ae6f02080
…
Removing intermediate container 6f1ae6f02080
```

- Sends build context (. directory) to docker daemon as an archive.
- Remote machine can build the image using local files.
- A container (bd3d4369aebc) is created from the base ubuntu image.
- The RUN command is executed and the container is committed to an image (6f1ae6f02080)
- Intermediate container (6f1ae6f02080) is removed.
- Docker builds up image and removes intermediate (temporary) layers during the process.

Caching system

- If we run the Dockerfile build again, it will happen almost instantaneously.
- Docker caches a snapshot of each intermediate image after each build step.
- While executing a build, Docker checks to see if it has already built the sequence of commands.
- You can force a rebuild using the **`docker build --no-cache`** command.
- The **`docker history <image>`** command lists all the layers within an image.

Dockerfile command: FROM

- **FROM**: defines the base image for the container.
  - Must be the first instruction after comments.
  - Lines starting with # are treated as comments.
  - Can be specified more than once to build multiple images.

```
FROM ubuntu
FROM training/sinatra
```

Dockerfile command: LABEL

- **LABEL**: create meta-data key-values for maintenance and documentation purposes.

```
LABEL maintainer=“Jason Baker”

LABEL version=“1.4.2”
```

Dockerfile command: CMD

- **CMD** : defines a default command to run when the container is started (if no other command is provided).
  - Can appear at any point in the Dockerfile.
  - Each CMD will override the previous one.

```
FROM ubuntu
RUN apt-get update
RUN apt-get install figlet
CMD figlet -f script hello
```

- We can always override the container CMD during startup by specifying a different command (in this case, bash):

```
$ docker run -t figlet bash
```

Dockerfile command: ENTRYPOINT

- **ENTRYPOINT** : specifies the command which absolutely must run every time the container is launched.

```
FROM ubuntu
RUN apt-get update
RUN apt-get install figlet
ENTRYPOINT ["figlet", "-f", "script"]
```

- Command line arguments are appended to ENTRYPOINT parameters.
- ENTRYPOINT can be located anywhere in the file.

```
$ docker run figlet awesome
```

Dockerfile command: COPY

- **COPY** : copies files from the build context into the container being built, COPY <from> <to>

- Create a file called hello.c in the container build directory:

```
int main () { puts("Hello, world!"); return 0; }
```

- Create new Dockerfile:

```
FROM ubuntu
RUN apt-get update
RUN apt-get install -y build-essential
COPY hello.c /
RUN make hello
CMD /hello
```

Docker file command: WORKDIR

- **WORKDIR** : sets the working directory for any RUN, CMD, ENTRYPOINT or COPY commands that follow it.

- Can be used multiple times in a Dockerfile.

```
WORKDIR /a
WORKDIR b
WORKDIR c
RUN pwd
```

- Output from this PWD command is /a/b/c

Dockerfile command: ENV

- **`ENV`** : sets an environment variable key to a value.
- Two formats:
  - ENV myName="John Doe"
  - ENV myName John Doe
- It is possible to override or set environment variables using **`docker run --env <key>=<value>`**

Dockerfile command: EXPOSE

- **EXPOSE** : informs Docker that the container process is available on the specified network ports at runtime.

- **Does not** automatically make the ports available to the host at runtime.
  - Need to use the –p or –P flags when running container.

Networking

- Containers can only have **private** IP addresses and services are exposed via ports.
- Docker supports port mapping to allow external machines to access services running in containers.

```
$ docker run –d –P nginx
```

- The –P publishes the container to make it reachable from other computers.

```
$ docker ps
CONTAINER ID        IMAGE     ... PORTS
a386440e6859        nginx         0.0.0.0:32769->80/tcp, 0.0.0.0:32768->443/tcp
```

Networking

- We can view the port mapping for a specific port in a container by running the command **`docker port <container ID> <port #>`**:

```
$ docker port a386 80
```

- It is possible to manually allocate port numbers to a container using **`docker run –d –p 80:80 nginx`**
  - The format is host port:container port

- Use the **`docker inspect`** command to find the IP address:

```
$ docker inspect --format '{{ .NetworkSettings.IPAddress }}'
a386
```

Networking

- A container can operate in a particular networking mode (called a driver).
- **Bridge** (default)
  - Container gets a virtual eth0 interface (172.17.0.0/16 private IP address).
  - Connected to Docker bridge.
  - Container can have own routes, iptables, etc.
- **Null**
  - Container has no network -- useful for isolated workloads.
- **Host**
  - Container access network interfaces of the host and binds to its ports.
  - Native performance.
- **Container**
  - Re-uses the network stack of another container and shares IP address, routes, iptables, etc.

Networking

- Docker uses the notion of networks to create a virtual network infrastructure between containers.
- Networks can be local or global and each network as an IP subnet.
- Use the **`docker network ls`** command to view the current networks setup on the host.
- Create a new network using the **`docker network create <name>`** command.
- Launch new containers on the new network:

```
$ docker run -dit --name db1 --net prod alpine sh
$ docker run -it --name web1 --net prod ubuntu
```

Volumes

- Any data created inside a Docker container disappears when the container is deleted.

- We use volumes to persist data outside of the container – they exist independently.

```
$docker run –d –v ~/html:/usr/share/nginx/html –p 8000:80 nginx
```

- The **–v** flag specifies a volume mount point
  - Format is [host-path]:<container-path>:[rw|ro]
  - Volumes are read-write by default.

Volumes

- We can also declare volumes inside a Dockerfile using the **VOLUME** instruction.

```
VOLUME /var/lib/postgresql
```

  - Where is the mapping? Volumes don't have to be anchored to a specific path! Use **`docker volume ls`** command to show volumes.

  - Also, we don't want to map volumes in a Dockerfile because of portability issues.

- Volumes bypass the Docker copy-on-write and have native I/O performance.

- Docker commits don't pull in volume data.

- Containers can share the same volumes using the –volumes-from flag:

```
$ docker run –it  --name alpha –v /var/log/ ubuntu bash
$ docker run --volumes-from alpha ubuntu bash
```

Docker Compose

- Dockerfiles can become fairly complex and can only specify configuration for one container.

- Docker Compose supports the building of multiple containers with a YAML file.
  - **`docker-compose.yml`**

```
version: '2'
services:
  web:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - .:/code
    depends_on:
      - redis
  redis:
    image: redis
```

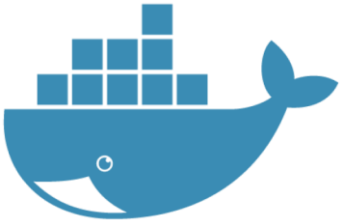
Troubleshooting Containers

- How do you access a container to troubleshoot it?

- Use the **`docker exec`** command to start a new process inside a running container.

```
$ docker exec –it web1 bash
```

Docker Hub



- We can upload images to the Docker Hub and make them publically or privately accessible.
- Docker Hub images can be utilized by multiple systems.
- Use the **`docker push username/imagename`** command to push images to Docker Hub.